Kathmandu University

Department of Computer Science and Engineering

Dhulikhel, Kavre

A Mini Project Report

on

"Optical Character Recognition"

Artificial Intelligence (Comp 472)

(For partial fulfillment of 4th year / 1st Semester in Computer Engineering)

Submitted by:

Dikshyant Adhikari (04)
Ajit Kumar Das (12)

Submitted to:

Mr. Sanjog Sigdel

Department of Computer Science and Engineering

Submission Date: July 7th 2025

Abstract

Optical Character Recognition (OCR) is a crucial application of Artificial Intelligence that enables machines to recognize and digitize printed or handwritten text from images. This project explores a CNN based approach using Python and TensorFlow to design a simple OCR system. The system processes input images, identifies characters, and outputs readable text. With wide ranging applications in document digitization, form processing, and license plate recognition, this system highlights the potential of deep learning in real-world text recognition tasks.

Keywords: *TensorFlow, Optical Character Recognition, Convolution Neural Network, Deep Learning.*

Table of Contents

1. Introduction

Optical character recognition (OCR) is a system that converts input text into machine-encoded format [1]. Initially developed to interpret printed documents, OCR systems have now become vital in many fields, including postal automation, banking, document digitization, and license plate recognition. In recent years, OCR has evolved to handle diverse inputs such as handwritten medieval manuscripts [2], ancient scripts, and real-time scene text from mobile devices or surveillance systems [3].

With the rise of deep learning, especially Convolutional Neural Networks (CNNs), the accuracy and robustness of OCR systems have significantly improved. Modern systems are capable of learning from large datasets without requiring handcrafted features, making them adaptable to variations in handwriting style, font, orientation, and noise [4].

In this mini project, we focus on developing an OCR system using CNN's to recognize alphanumeric characters (A–Z, 0–9). The goal is to preprocess character images, train a model to classify them, and deploy the system with a simple GUI that allows users to input images and get predictions in real time.

2. Objectives

The objectives of this project are to:

- To build an OCR system using CNN and TensorFlow.
- To recognize characters from input images.
- To evaluate the performance of the model using real-world samples.

3. Related Works

Early OCR systems were primarily based on template matching and handcrafted features. For instance, the **Tesseract** OCR engine, originally developed by HP and later open-sourced by Google, relied on connected components and character templates for recognizing printed text [5].

Figure: Google Tesseract OCR

With the rise of deep learning, Convolutional Neural Networks (CNNs) became the dominant approach for OCR. LeCun et al. introduced **LeNet-5**, one of the earliest CNN architectures, which showed excellent performance on digit recognition tasks like MNIST [6]. This laid the groundwork for modern OCR systems that can learn directly from raw pixels, removing the need for manual feature engineering.

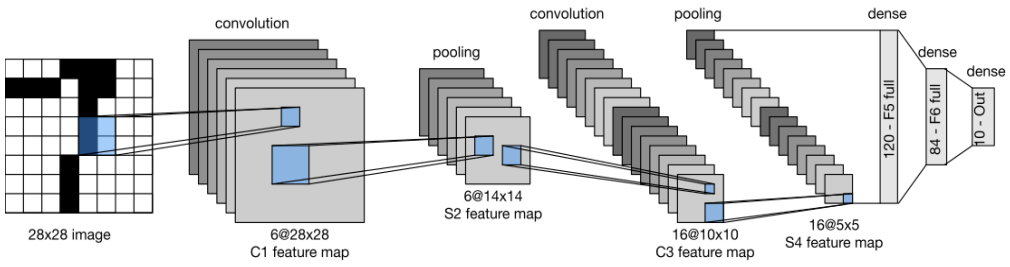


Figure : Architecture of LeNet-5 [6]

4. Methodology

Our OCR system is composed of four key modules that work together in a pipeline to transform input images into recognized characters. Each module plays a specific role in the character recognition process:

4.1 Training of the Model

The core of the OCR system is a Convolutional Neural Network (CNN) trained to classify character images. The model is trained on labeled grayscale images of size 64×64 pixels, covering digits (0–9) and uppercase letters (A–Z). The CNN consists of:

- Two convolutional layers (with ReLU activations and max pooling)
- One fully connected layer
- An output softmax layer to produce a probability distribution

4.2 Image Preprocessing

The raw input image is first converted to grayscale using OpenCV to simplify computation. The image is then resized to 64×64 pixels, one of the standard input sizes for the CNN model. This step ensures consistent input for the neural network.

```
img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

img_resized = cv2.resize(img, (64, 64))
```

4.3 Classification

Once trained, the CNN acts as a classifier. Given a new input image, the model computes the most likely character by outputting the index of the maximum probability. The predicted class index is then mapped back to a human-readable character using the label encoder.

4.4 Output Display

The final prediction is rendered to the user through a graphical interface built with Tkinter. The GUI allows the user to:

- Browse and select an image file
- View the image in a rescaled preview window
- See the predicted character displayed as text

The predicted character is displayed to the user. In the GUI version of the system, the image is shown in a window and the corresponding prediction is presented as a label below the image. The user can select new images and receive instant predictions.

5. System Architecture

5.1 Data Set

The dataset used in this project is the Standard OCR Dataset, publicly available on Kaggle [7]. It contains images of uppercase letters (A–Z) and digits (0–9) intended for training machine learning models in optical character recognition (OCR) tasks.

5.2 Model Architecture

The OCR system uses a Convolutional Neural Network (CNN) designed to classify 64×64 grayscale images of characters into one of 36 classes (digits 0–9 and uppercase letters A–Z). The model architecture consists of the following layers:

- **Input Layer:**
  Accepts a single-channel grayscale image with dimensions 64×64

- **Convolutional Layers:**

  Two convolutional layers are used, each followed by a ReLU activation function to introduce non-linearity, and max pooling layers to reduce spatial dimensions and retain important features. These layers automatically learn hierarchical image features such as edges and shapes.

- **Flatten Layer:**

  Converts the 2D feature maps into a 1D feature vector, preparing it for the dense layers.

- **Dense Layers:**

  Fully connected layers that learn higher-level representations and help in decision making. A dropout layer is included for regularization to prevent overfitting.

- **Output Layer:**

  A dense layer with softmax activation outputs a probability distribution over the 36 possible classes. The predicted class corresponds to the highest probability.

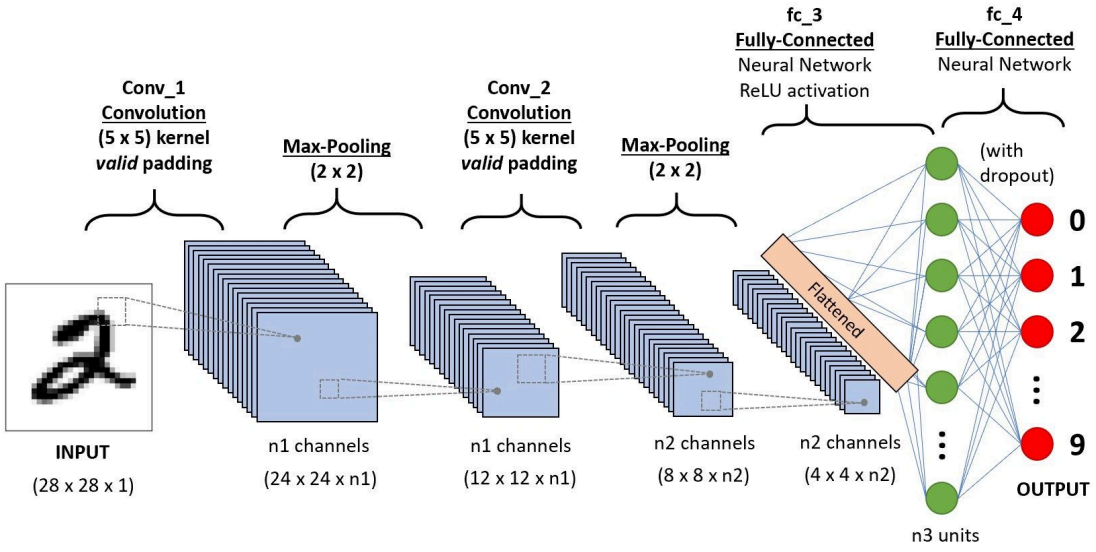
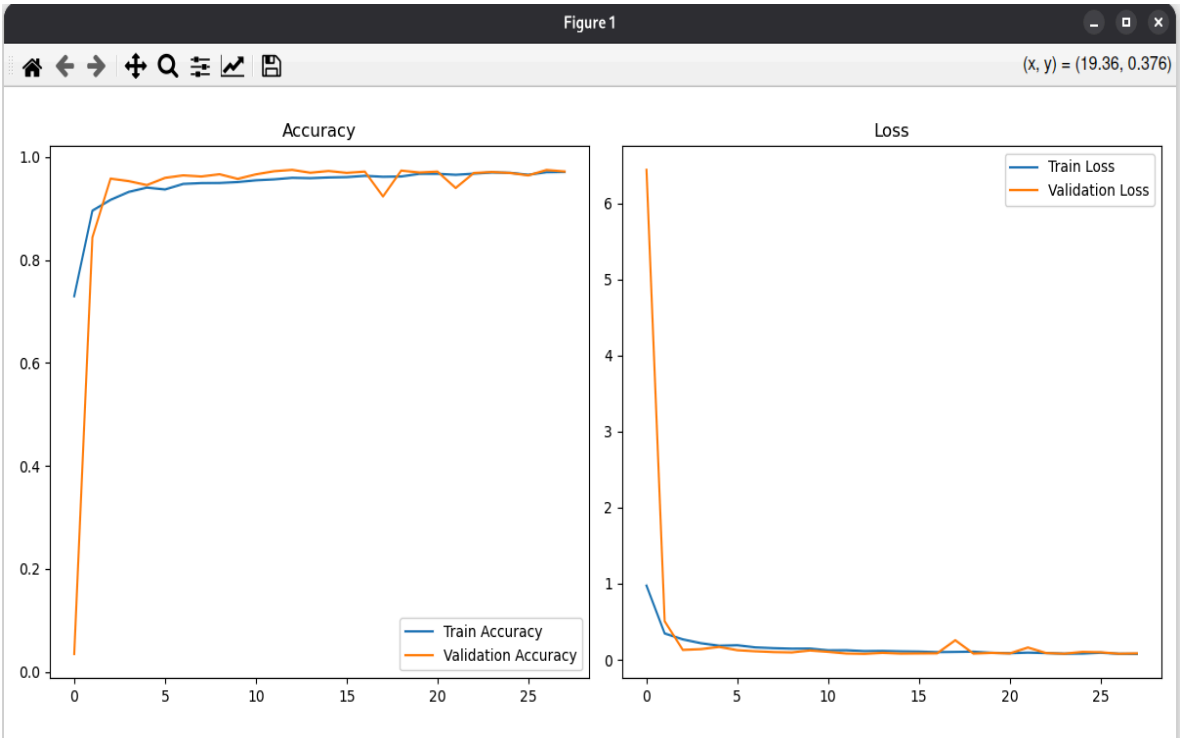


Fig : Convolutional Neural Network

5.3 Training Details

The CNN model was trained using a supervised learning approach on labeled grayscale character images. The following hyperparameters and configurations were used during training:

- **Language :** Python
- **Framework :** TensorFlow (Keras)
- **Model Type:** Sequential (Keras API)
- **Number of Epochs:** 30
- **Optimizer:** Adam (adaptive learning rate optimization)
- **Loss Function:** Categorical Cross-Entropy
- **Batch Size:** (32, default)
- **Activation Functions:**
    - ReLU in convolutional and dense layers
    - Softmax in the output layer
- **Input Shape:** (64, 64, 1) — grayscale image
- **Output Classes:** 36 (0–9, A–Z)



Model: "sequential"

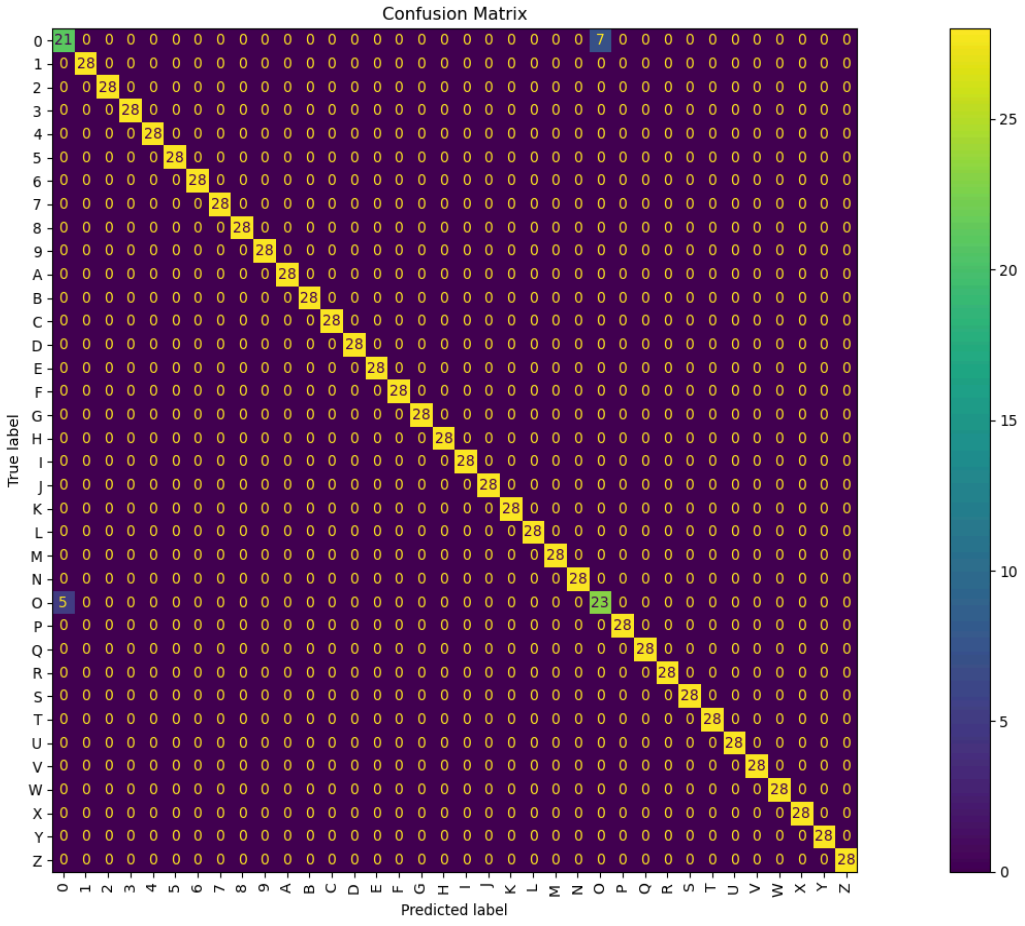
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 62, 62, 32)	320
max_pooling2d (MaxPooling2D)	(None, 31, 31, 32)	0
conv2d_1 (Conv2D)	(None, 29, 29, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_2 (Conv2D)	(None, 12, 12, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 6, 6, 128)	0
flatten (Flatten)	(None, 4608)	0
dense (Dense)	(None, 128)	589,952
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 36)	4,644

Total params: 687,268 (2.62 MB)
Trainable params: 687,268 (2.62 MB)
Non-trainable params: 0 (0.00 B)

6. Results

The Optical Character Recognition (OCR) system achieved excellent performance on the test dataset, with a Test Accuracy of 98.81% (Training accuracy of 97%). This high level of accuracy demonstrates the effectiveness of the convolutional neural network (CNN) in learning and generalizing character patterns from the training data.

A Confusion Matrix was generated to further analyze the model's predictions. Most character classes show very few misclassifications, indicating strong per-class performance. Minor confusion occurred between visually similar characters (e.g., ‘O’ and ‘0’), which is common in OCR systems.



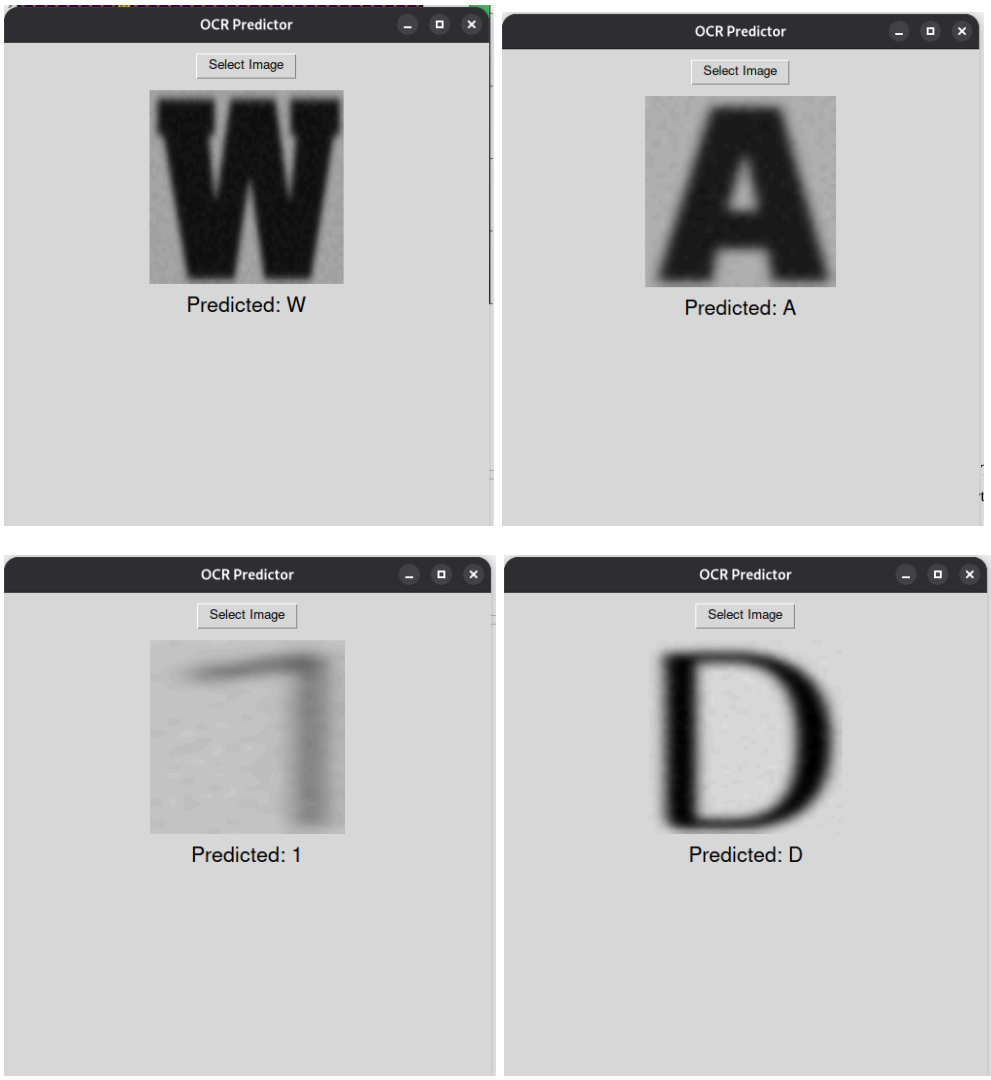


Figure : Some Screenshots

7. Conclusion

This project successfully demonstrates the application of Convolutional Neural Networks **(CNNs)** in building an OCR system capable of recognizing individual characters. By preprocessing grayscale images and training a deep learning model on 36 distinct classes (A–Z, 0–9), the system achieves a high test accuracy of **98.81%**, highlighting the effectiveness of the approach.

The use of a custom GUI further enhances usability, allowing real-time prediction of characters from image inputs. The model performs reliably even under moderate image noise or lighting variation, making it a strong foundation for more advanced OCR applications.

8. Future Work and Enhancements

To further enhance the system's capabilities and move toward real-world deployment, the following future directions are proposed:

1. **Word and Sentence Recognition:**
   Extend the model to recognize full words or entire lines of text using **CRNN** (Convolutional Recurrent Neural Network) or **RNN + CTC** (Connectionist Temporal Classification) frameworks.
2. **Multilingual Support**:
   Integrate and train the system to support multilingual text recognition, including complex scripts (e.g., Devanagari (Nepali), Chinese, Arabic).
3. **Application Deployment:**
   Convert the system into a web or mobile app using frameworks like Flask, Streamlit, or Flutter, enabling wider accessibility and real-time camera input.
4. **Handwriting**:
   Enhance the model to support cursive or difficult to read handwriting, which requires advanced segmentation and temporal sequence modeling.

9. References

[1] C. C. Tappert, C. Y. Suen, and T. Wakahara, ‘‘The state of the art in online handwriting recognition,’’ IEEE Trans. Pattern Anal. Mach. Intell.,vol. 12, no. 8, pp. 787–808, Aug. 1990, doi: 10.1109/34.57669.

[2] Stutzmann, D., & Puig, O. (2020). *Handwritten text recognition in medieval manuscripts using OCR and machine learning*. *Digital Scholarship in the Humanities*, 35(3), 505–521.

[3] Jaderberg, M., Simonyan, K., Vedaldi, A., & Zisserman, A. (2016). *Reading Text in the Wild with Convolutional Neural Networks*. *International Journal of Computer Vision*, 116, 1–20.

[4] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. (Chapter 9: Convolutional Networks)

[5] Smith, R. (2007). *An overview of the Tesseract OCR engine*. Google Inc. In ICDAR.

[6] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). *Gradient-based learning applied to document recognition*. Proceedings of the IEEE, 86(11), 2278–2324.

[7]Abhishek Jaiswal (2022). *Standard OCR Dataset*. Kaggle. https://www.kaggle.com/datasets/preatcher/standard-ocr-dataset